

TextWonder — Interview Preparation

Questions an engineer or AI interviewer will ask about this project — and how to answer them

This covers the project as it actually is, including its weak points. Interviewers find weak points; it is far better to raise them yourself than to be caught. Numbers below are measured from the real build — memorise the fact sheet.

Fact sheet — know these cold

226 tools	11 categories	2,636 pages built	10 languages
254 blog articles	~26s full build	110 tool components	100% client-side

Layer	Choice
Framework	Astro 6 (output: 'static')
Styling	Tailwind CSS v4 via @tailwindcss/vite
Hosting / CDN	Cloudflare Pages, deployed with wrangler
Edge logic	Pages Functions middleware (www→apex redirect, robots on preview)
PDF / image work	pdf-lib + PDF.js + Canvas API, all in-browser
Largest sections	Text 109 · PDF 28 · Dev 17 · Image 17 · Calc 15

1 — Opening questions

Q Tell me about TextWonder.

PROBING: can you scope a large project into 60 seconds without rambling?

- "A static site hosting 226 free browser-based utilities — text processing, PDF editing, image compression, developer tools, calculators — across 11 categories."
- "The defining constraint is that **every tool runs entirely client-side**. No file is ever uploaded. That is the product differentiator and it also means hosting costs almost nothing and there is no backend to attack."
- "It is built with Astro and pre-renders to 2,636 static pages, including a 10-language internationalisation layer."
- Then stop. Let them pick the thread.

Q Why did you build it? What problem does it solve?

- Existing free tool sites upload your file to a server to merge a PDF — a privacy problem for contracts, payslips, ID documents.
- They also gate features, add watermarks, or require signup.
- TextWonder does the work in the browser: no upload, no watermark, no account.

Be ready for: "How much traffic does it get?"

If you do not have strong numbers, say so plainly and pivot to what you *do* measure: pages indexed, Search Console impressions, Core Web Vitals. Never invent traffic figures — an interviewer may ask you to open the dashboard.

2 — Architecture & technology choices

Q Why Astro instead of Next.js or plain React?

PROBING: do you choose tools for reasons, or by habit/hype?

- "The site is 2,600 mostly-static content pages. Astro ships **zero JavaScript by default** and I only opt in per component — a React SPA would ship a runtime to every page that mostly displays text."
- "SEO is the entire acquisition channel, so pre-rendered HTML is non-negotiable. Astro does that natively without the SSR complexity."
- "Each tool is an isolated island of interactivity. That maps exactly onto Astro's islands model."
- Honest counterpoint to offer: "If the site had shared client state or user accounts, Next.js would be the better fit."

Q Walk me through what happens when someone requests a tool page.

- Nothing happens at request time — the HTML was generated at build. Cloudflare's CDN serves a static file from the nearest edge.
- At build, `src/pages/tools/[slug].astro` calls `getStaticPaths()` over the `TOOLS` array in `src/data/tools.js`.
- Each tool object carries its own SEO payload: `metaTitle`, `metaDescription`, `h1`, `descriptionLong`, `useCases`, `howItWorks`, `faqs`, `relatedTools`.
- A slug→component map resolves which interactive widget to mount; the layout wraps it with schema markup, breadcrumbs and related links.

Q Why keep tool content in JavaScript data files instead of a CMS or Markdown?

- "Content is highly structured — every tool has exactly the same nine fields. That is a schema, not prose, so a typed object is a better fit than free-form Markdown."
- "It makes bulk operations trivial: I can audit or reshape all 226 tools with a script."
- Honest cost: "It is now ~9,000 lines of hand-maintained JS. If a non-technical editor ever needed to write content, I would move to a headless CMS or content collections."

Q Your build produces 2,636 pages in about 26 seconds. What happens at 50,000?

PROBING: do you understand the limits of your own design?

- "Build time is roughly linear, so ~50k pages is on the order of eight minutes — tolerable but past the point of comfort."
- "The fix is not a faster machine, it is changing the rendering mode: Astro supports hybrid/on-demand rendering, so I would pre-render the high-traffic pages and render the long tail on demand with CDN caching."
- "Incremental builds are the other lever — only rebuild what changed."

3 — Internationalisation (your strongest technical story)

Q How did you implement multi-language support?

- A registry in `src/i18n/` holds locale config, UI chrome strings, and per-locale tool translations keyed by the **English** slug.
- Keying on the English slug is the important detail — it lets localized routes reuse the exact same tool components while the URL segment changes.
- Three dynamic routes under `src/pages/[locale]/` generate the home, section index and tool pages for every language.

```
EN /tools/word-counter/  
ES /es/herramientas-texto/contador-de-palabras/  
FR /fr/outils-texte/compteur-de-mots/  
RU /ru/tekstovye-instrumenty/schetchik-slov/
```

Q Why translated URL slugs instead of `?lang=fr` or a subdomain?

- "Query parameters are the worst option — Google often treats them as the same page, so you get no separate ranking."
- "Subdomains split domain authority; every language starts from zero. Subdirectories inherit the authority of the root domain."
- "Translated slugs put the keyword in the URL, and more importantly they make the page look native to a French user in the results list, which lifts click-through."

Q Explain hreflang. What breaks if you get it wrong?

PROBING: do you actually understand it or did you copy a snippet?

- "It tells Google that several URLs are the same page in different languages, so it can serve the right one instead of treating them as competitors."
- "**It must be bidirectional.** If the English page does not list the French one, Google discards the whole cluster. That is why the English layouts also emit the cluster, not just the translated pages."
- "Each cluster also needs exactly one `x-default` for users whose language you do not cover."

Q Why do Japanese and Arabic pages use English URL slugs when Spanish uses native ones?

- "Deliberate, per script. Latin-script locales get native slugs. German transliterates umlauts (ö→oe)."
- "Hindi and Russian use ASCII transliteration; Japanese and Arabic use English-descriptive slugs — that is the prevailing convention on tool sites in those markets."
- "Non-ASCII paths also break on Windows checkouts and during CDN upload. The native keywords still appear in the title, H1 and body, which is where they carry ranking weight — the URL is a very minor factor."

Q You build 2,260 localized pages but only 243 are indexable. Explain.

PROBING: this is the best judgment question in the project. Take your time.

- "Only 20 tools per language are actually translated so far. The rest render with localized navigation but English tool copy."
- "Those fallback pages exist so the catalogue never looks half-empty in any language — a French user still sees all 226 tools and every one works."
- "But they are marked `noindex`, excluded from the sitemap, and excluded from every hreflang cluster. Those three signals must always agree."
- "Why: I measured them. An untranslated French page has a **byte-identical title, H1 and meta description** to the English original. Indexing 1,854 of those is textbook duplicate content — Google would consolidate them and could devalue the English page."
- Closing line: "And it would gain nothing. A French query cannot match an English title. So it is risk with zero upside — the real fix is translating more tools, not flipping a flag."

Q How do you handle right-to-left for Arabic?

- `dir="rtl"` on `<html>`, driven by the locale registry.
- Localized components use CSS **logical properties** (`ms-`, `me-`, `ps-`, `start-`) which flip automatically; shared English components that still use physical `left/right` get targeted overrides.
- Nuance worth mentioning: "Text inputs and code blocks are forced back to LTR — user content is usually not Arabic, and mirroring it would scramble URLs and JSON."

4 — SEO engineering

Q What structured data do you emit, and why?

- `WebApplication` on tool pages, `BreadcrumbList`, `FAQPage`, `CollectionPage` on section indexes, `Article` on blog posts.
- "FAQPage is the highest-value one — it can win extra vertical space in results."
- Localized pages also set `inLanguage` so the markup matches the page language.

Q Canonical, hreflang, noindex — when do you use each?

Tag	Says	Used here for
canonical	"this is the master copy of this content"	Self-referencing on every page
hreflang	"same content, different language"	Linking the 10 language versions
noindex	"do not list this at all"	Untranslated fallback pages

Trap to avoid out loud: "You should not point a canonical at another page *and* noindex it — the signals contradict. Fallback pages are noindex with a self-canonical."

Q How does your sitemap decide priority?

- Derived, not hardcoded: home 1.0, section index 0.9, tool page 0.8, everything else 0.6. Localized equivalents sit one notch lower.
- The section list is imported from the routing config, so a new section cannot drift out of sync with the sitemap.
- `noindex` pages are filtered out entirely — sitemap and robots directives must not disagree.

5 — Debugging stories (rehearse at least two)

Structure every debugging answer as: symptom → what I first assumed → how I proved the real cause → the fix → what stops it recurring.

Q Tell me about a bug that took you a long time to find.

Symptom. Every one of 254 blog posts rendered as an undifferentiated wall of text — headings the same size as body copy, bullet lists with no bullets.

Wrong first assumption. That the content was broken. It was not — the HTML had correct `<h2>` and `<ul>` tags.

Real cause — two independent failures at once:

- The layout styled content with Tailwind Typography `prose-*` classes, but that plugin **was never installed**, so every one of those classes was inert.
- The fallback `<style>` block had correct rules, but Astro **scopes component styles** to that component's own markup. Blog copy arrives through `<slot />` from the page, so the scoped selectors never matched it.

With both dead, Tailwind's Preflight reset applied: `h2` fell back to `font-size: inherit; margin: 0`.

Proof, not guesswork. I rendered the page in headless Chromium and read the computed styles: `h2` was 17px/inherit/0 margin. After the fix, 25.6px/700/48px.

Fix. Moved typography to a global stylesheet under a `.blog-article` class — one change, all 254 pages.

Lesson to state: "Two silent failures masked each other. Neither threw an error, which is why reading computed values beat reading code."

Q Tell me about a bug with real consequences that you almost missed.

"While auditing all 254 blog pages with a script, I found two that nested `<BlogLayout>` inside `<BaseLayout>` — but `BlogLayout` already renders `BaseLayout`."

- Each page emitted **two complete** `<html>` and `<head>` documents.
- Two canonical tags — and the inner one pointed at the **homepage**, actively telling Google the article was a duplicate of the home page.
- A second, empty meta description; and the byline read "Invalid Date" because the post passed `publishedDate` while the layout reads `date`.

Why it matters: "Visually the page looked fine. It was only broken in ways that hurt search. That is why I audit the built HTML programmatically rather than eyeballing pages."

Q Give me an example of when you nearly made things worse.

PROBING: intellectual honesty and verification discipline. Very strong answer if delivered calmly.

"My audit script flagged four blog posts as containing leaked Markdown asterisks. If I had bulk-fixed them I would have corrupted three."

- Two were **intentional profanity masking** ("F***ing") in slang explainer articles.
- One was **Python comments inside a code block**.
- Only one was a genuine leak.

Lesson: "A regex flags candidates, not defects. I now treat scanner output as a list to inspect, never a list to auto-fix."

Q How do you verify a change across thousands of pages?

- Scripted assertions over the built output — counts of `<html>`, canonical, description, `h1` per page.
- A link crawler over all localized pages: 67,986 internal links checked, zero broken.
- Headless Chromium for computed styles and screenshots when the question is visual.
- Live verification after deploy — because a green build does not prove what the CDN is serving.

6 — Performance, security, scale

Q Your homepage HTML is 378 KB. Is that acceptable?

PROBING: will you defend a weak number or own it? Own it.

- "No, it is heavier than I would like, and I know exactly why: the page renders all 226 tool cards for the category dashboard and inlines a JSON search index. Inline script is about 18% of the bytes."
- "It gzips well because it is repetitive HTML, and it is a static file from a CDN edge, so real-world impact is smaller than the raw number suggests."
- "The fix I would make: render only the first tab's cards and fetch the rest on demand, and move the search index to a separate cached JSON file so it is not re-downloaded with every HTML page."

Q There is no backend. What is your security model?

- "No server means no server-side attack surface — no database, no auth, no file storage, no upload endpoint to exploit."
- "The residual risk is client-side: XSS. The main place I build HTML from data is the search dropdown, so that output is escaped."
- "Third-party scripts (analytics, ads) are the largest remaining trust dependency."
- Improvement to volunteer: "I would add a Content-Security-Policy header via the Pages middleware — that is a genuine gap."

Q A user uploads a malicious PDF. What happens?

- "It never leaves their machine, so it cannot compromise my infrastructure — there is nothing to compromise."
- "It is parsed by PDF.js inside the browser sandbox. The realistic failure mode is a crash or hang of their own tab, not a breach."
- "Compare that to a server-side competitor, where a malicious file reaches a parser on their infrastructure and every user's documents sit in one bucket."

Q How would you add user accounts and saved history?

PROBING: can you extend an architecture without wrecking its premise?

- "Carefully — privacy is the product, so I would not quietly start sending documents to a server."
- "Keep the tools client-side. Store history in IndexedDB by default, local to the device."
- "If sync is required, make it explicit opt-in and encrypt client-side so the server stores ciphertext it cannot read."
- "Astro static plus Cloudflare Workers/D1 for the account API — the content stays static, only the account layer is dynamic."

7 — Weak spots: raise these before they do

These are real gaps in the project. An interviewer who looks will find them. Naming them first reads as engineering maturity; being caught reads as not knowing your own codebase.

Gap	How to answer
No automated tests. package.json has only dev/build/preview.	"The honest answer is there is no test suite, and for 110 interactive tools that is the biggest risk in the codebase. Astro's build catches broken imports and routes, and I audit built HTML with scripts, but neither proves the word counter counts correctly. First thing I would add: Vitest unit tests over the pure transform functions, then Playwright smoke tests on the top tools. Playwright is already a dependency."
No CI/CD. Deploys are manual wrangler commands.	"Manual, and it has bitten me — deploying to the wrong branch publishes to preview instead of production. A GitHub Action on push to master would remove that whole class of error."
Tool UI labels are English in all 10 languages.	"All SEO-bearing copy is translated, but in-widget labels like 'Your Text' are not. It needs a locale prop threaded through ~110 components. I sequenced the SEO layer first because that is what actually acquires users."
Only 20 of 226 tools translated per language.	"Deliberate. Translating the long tail into nine languages is a large investment with unknown return, so I shipped a measurable wave first and plan to let Search Console data choose the next one."
9,000 lines of hand-maintained data files.	"Fine at 226 tools with one maintainer. It would need Astro content collections with schema validation before a second person touched it."
No error monitoring.	"If a tool throws in a user's browser I have no idea. Sentry or a lightweight error beacon is the obvious next addition."

8 — Behavioural questions, answered through this project

Q Tell me about a technical decision you changed your mind about.

- "I originally refused to generate a localized page unless it was fully translated — clean from an SEO standpoint."
- "Then I looked at it as a user: the Portuguese site advertised 20 tools while the English one had 226. It looked broken."
- "I inverted it — build the whole catalogue in every language for users, but mark untranslated pages noindex and keep them out of the sitemap and hreflang clusters."
- "The lesson: I had optimised one dimension, search, and let it damage another, user trust. The right answer served both."

Q How do you decide what to work on when everything seems important?

- "I ask what is broken versus what is merely incomplete. Blog posts rendering as a wall of text was broken and affected 254 pages — that outranked adding a new tool."
- "Then leverage: one CSS change fixed every blog page. I prefer fixes at the layer where one change propagates."

Q Someone insists on a change you believe is wrong. What do you do?

- "Bring evidence rather than opinion. When indexing all untranslated pages came up, I measured them and found the titles and H1s were byte-identical to existing pages."
- "That reframes it from 'I disagree' to 'here is what will happen'. And I paired it with the alternative that achieves their actual goal — translating high-volume tools."
- "If they still want it after seeing the data, it is their call and I would implement it — but the risk is on the record."

Q What would you do differently if you started over?

- "Design the i18n data model on day one. Retrofitting it meant extracting slug→component maps that had been duplicated across ten route files."
- "Write tests for tool logic from the start — retrofitting tests across 110 components is far harder than writing them alongside."
- "Set up CI before the site got big enough for manual deploys to be risky."

9 — Rapid-fire technical checks

Question	One-line answer
SSG vs SSR vs CSR — when each?	SSG for content that changes rarely; SSR for per-request personalisation; CSR for app-like state behind a login.
What is Tailwind Preflight?	Its CSS reset — it strips default heading sizes and list markers. It caused the blog bug.
Why does Astro scope styles?	To stop component CSS leaking. The trap: scoped rules do not reach <code><slot /></code> content.
What is <code>getStaticPaths()</code> ?	Tells Astro which pages to generate from a dynamic route at build time.
Core Web Vitals?	LCP (load), INP (responsiveness), CLS (visual stability). Static + CDN helps LCP most.
Why is <code>x-default</code> needed?	Fallback for users whose language has no version — points at English here.
robots.txt vs noindex?	robots.txt blocks crawling; noindex blocks listing. Blocking a page you want de-indexed backfires — the crawler can't see the noindex.
Why hash asset filenames?	Cache-busting — enables immutable long-lived caching.
How does the PDF merge work?	pdf-lib reads both files into memory in the browser, copies page objects into a new document, serialises to a Blob.
Biggest scaling bottleneck?	Build time and hand-maintained content, not traffic — static files on a CDN scale essentially free.

10 — Questions to ask them

- "How do you balance shipping speed against test coverage here — what does 'done' mean on your team?"
- "How much say does an engineer have in product or technical direction?"
- "What does your deploy pipeline look like, and how quickly can you roll back?"
- "What is the most painful piece of technical debt in the codebase right now?"
- "What would a strong first three months look like in this role?"

If asked how much of it was AI-assisted — answer directly. Modern interviewers ask, and evasiveness is far more damaging than the answer. Say what you used tools for, then demonstrate ownership by explaining a decision and its trade-off in your own words — the noindex/duplicate-content call, or why translated slugs beat query parameters. Understanding the *why* is what they are actually testing, and that is knowledge you can defend.

Two habits worth naming explicitly: "I verify against the built output rather than trusting the source," and "I check live after deploying, because a green build does not prove what the CDN is serving." Both are things a lot of candidates never mention.

TextWonder interview preparation · figures measured from the production build (226 tools, 2,636 pages, 10 locales) · review the Fact sheet and Weak spots sections last before an interview.